

Deep Reinforcement Learning Model for Robust Travel Salesman Problem via Multiobjective Optimization Framework

Anonymous Authors

Abstract—Deep Reinforcement Learning (DRL) has gained significant attention for its ability to solve combinatorial optimization (CO) problems, including the Traveling Salesman Problem (TSP). While traditional approaches assume known and deterministic cost coefficients, real-world scenarios often involve uncertainty, such as traffic conditions or varying travel times. To address this, we introduce a Multiobjective Robust Combinatorial Optimization (MO-RCO) model that incorporates robustness as an additional objective. By transforming a classical robust combinatorial optimization (RCO) problem into a multiobjective framework, MO-RCO allows decision-makers to explore optimal solutions via a Pareto front, while utilizing pre-trained multiobjective models to estimate robust solutions for new uncertain instances. This enables our method to balance optimality and robustness and provide a scalable solution for new RCO problems. Our results demonstrate the efficiency of MO-RCO in solving the min-max TSP with budget uncertainty, showcasing its potential for real-world applications in robust decision-making.

Index Terms—Robust Combinatorial Optimization, Multiobjective Optimization, Deep Reinforcement Learning, Traveling Salesman Problem

I. INTRODUCTION

Recently, the remarkable performance of deep reinforcement learning models (DRL) in solving combinatorial optimization (CO) problems has attracted widespread attention. Compared to traditional heuristic methods, DRL models demonstrate strong generalization and considerable efficiency, even when tackling large-scale problems. Furthermore, the Traveling Salesman Problem (TSP) has become the focus of learning-based optimization algorithms due to their inherent structural characteristics.

In [1], a neural architecture is firstly proposed to solve TSP by predicting the sequence. In [2], the actor-critic network is used to improve the training process. Since then, DRL approaches with attention mechanism have been widely studied [3], [4]. In addition, not only single-objective problems, but also multi-objective problems (MOP) can be effectively solved by DRL-based algorithms [5]–[8].

In the classical CO paradigm, the cost coefficients in the objective functions are typically precise and known. However, uncertainty is widely present in real-life scenarios. In the context of the TSP, for instance, factors including weather conditions, traffic situations, and the traveler’s status can affect the time spent on each edge of the route, thereby increasing the travel cost. When costs are uncertain, the possible discrepancy between the actual cost values and the nominal cost values

makes those idealized models no longer accurate. This practical need naturally leads to the research of robust CO models, which seek the optimal solution in the worst-case scenario.

Robust combinatorial optimization (RCO) was proposed by Mulvey [9] and Ben-Tal [10] and developed by [11]. RCO has several established approaches for defining robustness, including the min-max criterion, the min-max regret criterion, and adjustable robustness, etc [12]–[14]. The selection of the robustness criterion directly influences the form of the objective and has a significant impact on the complexity of the problem. For the robust TSP (RTSP), several works have been made in recent years. Montemanni et al. proposed an exact algorithm for the RTSP using the min-max regret criterion [15]. Chassein and Goerigk introduced a recoverable robust model that allows the modification of a limited number of edges after the scenario is known [16]. Lu et al. proposed a heuristic method based on simulated annealing and the LKH algorithm to solve the RTSP with the min-max regret criterion and interval uncertainty sets [17]. Hasegawa et al. introduced a heuristic edge generation algorithm to address this problem [18]. Xiao et al. introduced a DRL-based model to solve regret problems with budget uncertainty [19].

RCO generally considers the worst-case scenario, and the solutions generated by the model tend to be too conservative. Yet, the actual situations might not always be so extreme. When the degree of uncertainty is not as severe, overly conservative solutions obtained with a pessimistic approach are far from optimal. Therefore, decision-makers need to make a trade-off between the optimality and the robustness of the solution. A notable method for addressing this issue is the regret approach [20], which evaluates a solution not by its worst-case value but by comparing it with the best result that could have been achieved under each scenario. However, this method requires the additional computation of the optimal tour for each scenario, which increases computational complexity. Additionally, the obtained solution may still fail to effectively balance optimality and robustness. This brings to mind that MOP methods are capable of searching for multiple Pareto-optimal solutions in a single run [21], [22], offering a more flexible approach to addressing these trade-offs. Currently, solving RTSP mainly relies on traditional optimization methods, while DRL has rarely been applied to this domain. Meanwhile, as mentioned before, DRL has achieved success in MOP. Thus, designing a DRL structure specifically suited for RCO seems to be a natural direction.

Considering the above, we propose to incorporate robustness as a new objective and extend RCO into a multiobjective combinatorial optimization (MOCO) problem. The Pareto set generated by this model can be used by decision-makers for trade-off selection. Moreover, for new problems with uncertainty parameters (such as budget and deterioration levels) that have not been trained on, we propose a method that leverages an already trained multiobjective model to estimate the robust solution for the new problem based on the solutions of the subproblems. This also demonstrates the model's generalization ability, enabling it to assist decision-makers in various uncertain scenarios.

In this study, we focus on min-max TSP with budget uncertainty. The features of MO-RCO are as follows.

- By introducing robustness as a new objective, MO-RCO transforms RCO into a MOCO problem and solve it through DRL methods. The Pareto set generated by the model provides decision-makers with a set of trade-off solutions to choose from.
- MO-RCO can be used to solve multiobjective robust combinatorial problems. It addresses the conflict between the trade-off of each objective and the balance between optimality and robustness, which arises when solving them separately.
- For new problems with uncertainty parameters that have not been trained, MO-RCO can utilize an already trained multiobjective model to estimate robust solutions by directly using the solutions of subproblems.

The remainder of this paper is organized as follows. Section II briefly reviews the fundamentals. Section III presents the details of our proposed method. The experimental results are provided and analyzed in Section IV. The final section concludes the paper.

II. BACKGROUND

A. Robust TSP

The intuitive interpretation of the TSP is that one needs to follow a cyclic tour through n cities, minimizing the cost function. In order to facilitate subsequent processing, the TSP can be stated as:

$$\min_{x \in \Omega} \sum_{i=1}^{n-1} c_{x_i, x_{i+1}} + c_{x_n, x_1} \quad (1)$$

where Ω represents the decision space, with the solution $x = (x_1, \dots, x_n)^T$ being a permutation, and c_{ij} is the travel cost between city i and city j . For convenience, we denote (1) as a nominal TSP, and denote $\{(i, j) | i, j = 1, \dots, n, i \neq j\}$ as E .

In the min-max robust formulation of the problem, the costs c_{ij} are uncertain. In general, the RTSP can be stated as:

$$\min_{x \in \Omega} \max_{c \in U} \sum_{i=1}^{n-1} c_{x_i, x_{i+1}} + c_{x_n, x_1} \quad (2)$$

where U represents the uncertainty set of distance matrices, also referred to as scenarios.

B. Budget Uncertainty Set

The most common uncertainty set of RTSP is the box uncertainty set, where c_{ij} falls in an interval $[\underline{d}_{ij}, \bar{d}_{ij}]$ for each edge (i, j) . Such an uncertainty set is often criticized for being overly conservative, as the cost for each edge of the tour is assumed to always take the upper bound of the interval. As a result, the budget uncertainty set, introduced by Bertsimas and Sim [11], commonly referred to as the Γ -scenario uncertainty set, is frequently employed to mitigate the conservatism of the model. The budget uncertainty set is defined as $U = \{c | c_{ij} = \underline{d}_{ij} + \xi_{ij}(\bar{d}_{ij} - \underline{d}_{ij}), \sum_{(i,j) \in E} \xi_{ij} \leq \Gamma, \xi_{ij} \in [0, 1], \forall (i, j) \in E\}$. Note that when $\Gamma = 0$, the model is equivalent to that of the nominal problem, and when $\Gamma \geq n(n-1)$, it defaults to a box uncertainty model.

It is important to highlight that such a budget specification is not sufficiently effective in the TSP. In fact, when the budget $\Gamma \geq n$, since a tour contains only n edges, the cost of all edges can reach the upper bound of the interval in the worst-case scenario. Therefore, we need to apply the uncertain budget only to the edges selected in the solution x , which can be stated as:

$$U = \{c | c_i = \underline{d}_i + \xi_i(\bar{d}_i - \underline{d}_i), \sum_{i=1}^n \xi_i \leq \Gamma, \xi_i \in [0, 1], i = 1, \dots, n\} \quad (3)$$

where $\cdot_i$ is the shorthand for the edge starting from x_i .

C. Multiobjective Optimization

The mathematical formulation of a MOP with m objectives and n decision variables for minimization can be written as:

$$\min_{x \in \Omega} F(x) = (f_1(x), \dots, f_m(x)) \quad (4)$$

where Ω is the decision space. When objectives conflict, no single solution can be optimal for all objectives. The optimality of a MOP is typically defined by Pareto dominance. The set of all optimal solutions is referred to as the Pareto set (PS) in the decision space and the Pareto front (PF) in the objective space.

D. Deep Reinforcement Learning

In the past decade, significant achievements have been made in the field of DRL. Since the introduction of the Pointer Network [1], which utilizes a sequence-to-sequence model to select nodes sequentially based on the current partial solutions, many reinforcement learning models with policy gradients have been shown to effectively overcome the challenges that models with supervised training must face [2], [3]. Specifically, DRL models typically consist of two neural networks: an encoder and a decoder. The model compresses the input sequence into a vector using the encoder, selects the most likely next city index through the attention mechanism, and finally, the decoder unfolds the resulting high-dimensional vector.

III. MO-RCO

In this section, we propose to solve the RTSP with budget uncertainty in a DRL-based framework MO-RCO. By transforming RCO into MOCO, MO-RCO helps find the tour in an end-to-end manner. Specifically, MO-RCO outputs a Pareto set of tours that balance optimality and robustness, and provides a robust solution for a new problem. The main algorithmic framework of MO-RCO is illustrated in **Algorithm 1** and Figure 1. III-A introduces the method to transform objective, III-B describes the DRL model, III-C describes the calculation of worst-case tour length f^{wc} , and III-D introduces how to indicate solution for a new problem.

A. Objective Transformation

For single-objective robust combinatorial optimization problems, we introduce robustness as an additional objective, thereby forming a bi-objective robust combinatorial optimization problem. For multiobjective robust combinatorial optimization problems with m objectives, we similarly introduce robustness to form an optimization problem with $m + 1$ objectives.

In particular, for the initial single-objective RTSP, it is transformed into:

$$\begin{aligned} \min_x f_1(x) &= \sum_{i=1}^{n-1} c_{x_i, x_{i+1}} + c_{x_n, x_1}, \\ \min_x f_2(x) &= \frac{f_1^{wc}(x) - f_1(x)}{f_1(x)} \end{aligned} \quad (5)$$

where $f_1^{wc}(x)$ refers to $\max_{c \in U} \sum_{i=1}^{n-1} c_{x_i, x_{i+1}} + c_{x_n, x_1}$ in (2), representing the maximum value of the objective function under uncertain scenarios. In fact, it can be replaced by other forms, depending on different criteria of robustness.

For the initial multiobjective RTSP with m objectives, it is transformed into:

$$\begin{aligned} \min_x f_k(x) &= \sum_{i=1}^{n-1} c_{x_i, x_{i+1}}^k + c_{x_n, x_1}^k, \quad k = 1, \dots, m, \\ \min_x f_{m+1}(x) &= \sum_{k=1}^m \lambda^k \frac{f_k^{wc}(x) - f_k(x)}{f_k(x)} \end{aligned} \quad (6)$$

where $f_k(x)$ and $f_k^{wc}(x)$ refer to the k th objective of the original multiobjective problem and the maximum value under uncertain scenarios. It is noteworthy that the multiobjective robust combinatorial optimization discussed herein is a novel formulation, introduced for the first time by the authors. In fact, the additional objective does not necessarily have to take this form. We have designed it this way to introduce contradicting components between the objectives, which helps to enhance the training process.

Compared to the approach that handles the min-max nested problem, this transformation results in a simplified form for all objectives except the final one, thereby reducing computation. Furthermore, the application of this transformation in MOP carries a significant practical implication. Although each objective function $f_k(x)$ in the MOP is determined by uncertain

scenario $d^k \in U_k$, these scenarios are not independent of one another. Specifically, the worst-case scenarios d^k within each objective are often not achievable simultaneously. Consequently, the overall worst-case scenario must be considered by integrating all uncertain scenarios. In other words, from the perspective of decision-making in the internal maximization problem (aims to maximize cost), the interdependency, and even conflict, between different objectives necessitates a *trade-off* consideration of the uncertainty scenarios for each objective.

B. Model

We propose to use a DRL-MOA framework [6] as our basic model (see **Algorithm 2**). .

Algorithm 1: The Framework of MO-RCO

Input: Robust TSP instance (tour indices, the number of objectives M , distance matrix $\underline{d}, \bar{d}$), the degree of uncertainty (budget Γ), and the number of subproblems N .

Output: A set of nondominated robust solution (tour indices); the solution indicated by method.

- 1 Initialize weight vectors of objectives $\{w^1, w^2, \dots, w^N\}$;
 - 2 Generate weight vector of uncertainty $(\lambda_1, \dots, \lambda_k)$;
 - 3 Add new objective (robustness):

$$f_{M+1}(x) = \sum_{k=1}^M \lambda_k \frac{f_k^{wc}(x) - f_k(x)}{f_k(x)};$$
 - 4 **for** $j \leftarrow 1$ **to** N **do**
 - 5 Train / Test Model;
 - 6 **end**
 - 7 **return** *nondominated solutions*;
 - 8 **if** $M = 1$ & *uncertainty parameter* $\Gamma' == \text{untrained}$ **then**
 - 9 Indicate subproblem j for the new problem (Γ', d') ;
 - 10 $x' \leftarrow$ solution of subproblem j ;
 - 11 **end**
 - 12 **return** x' ;
-

1) *Decomposition and Parameter Transfer Strategy:* The MOP is decomposed into several scalar optimization subproblems using a well-established decomposition technique, such as the following Weighted Sum approach.

$$F(x) = \sum_{k=1}^m w_k f_k(x) \quad (7)$$

Each subproblem is modeled as a neural network. The neighborhood-based parameter transfer strategy is adopted to solve these subproblems collaboratively [22]. This strategy ensures that parameters from neighboring subproblems, which share similar weight vectors, are transferred during the training process, facilitating faster convergence and reducing training time. The DRL-MOA framework leverages this strategy to optimize the model parameters across all subproblems efficiently.

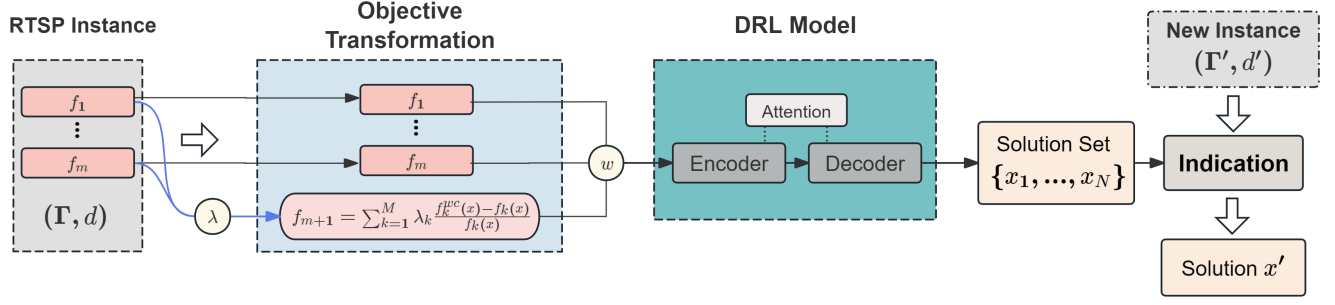


Fig. 1: The framework of MO-RCO. MO-RCO consists of three parts: the objective transformation, the DRL model, and the solution indication.

2) *Encoder and Decoder*: For each subproblem, a modified Pointer Network is applied to represent the problem. The architecture includes two main components: the encoder and the decoder. The encoder is responsible for transforming the input city attributes (such as coordinates and costs) into a high-dimensional vector using a 1-dimensional convolution layer. The decoder, in turn, uses a RNN to sequentially select the next city in the tour based on previously visited cities as follows (see line 4).

$$p_\theta(x | S) = \prod_{t=1}^n p_\theta(x_t | S, x_{1:t-1}) \quad (8)$$

The model also employs an attention mechanism that assigns importance to each city.

3) *Reinforcement Learning Training Method*: The DRL-based framework uses the Actor-Critic network to train the model. The actor network generates a probability distribution for selecting the next city, while the critic network evaluates the expected reward given the current state. The training is conducted in an unsupervised manner, where instances of the MOTSP are generated from various distributions, such as the Euclidean coordinates of cities. The actor network is updated using policy gradients based on the rewards obtained for each tour, while the critic network is trained to approximate the expected reward more accurately. This iterative training process enables the model to learn optimal city sequences that minimize the objective functions.

C. Worst-case Tour length of RTSP

For the RTSP, we need to modify the reward in reinforcement learning to capture the tour length under the worst-case scenario. Explicitly, we introduce distance uncertainty into the the tour length and incorporate a maximization problem by computing the maximum value of ξ at each iteration. This maximization of the objective function (f^{wc}) will lead to the worst-case reward, transforming the problem into a min-max formulation.

The top-k operation (see line 2 in **Algorithm 3**) is derived from the following worst-case scenario lemma [23]. The lemma mentions that allocating the worst-case budget

Algorithm 2: Model Training

Input: Training set S , batch size B , the number of subproblems N , weight vector w^j , and the maximal number of training iterations l_{max} .

Output: Model parameter θ^* .

```

1 Initialize model  $\theta$ ;
2 for  $iteration \leftarrow 1$  to  $l_{max}$  do
3    $s_i^j \sim \text{SampleInstance}(S), \forall i \in \{1, \dots, B\}$ ;
4    $\pi_i^j \sim \text{SampleSolution}(p_\theta(\cdot | s_i)), \forall i \in \{1, \dots, B\}$ ;
5    $\nabla L(\theta) \leftarrow$ 
6      $\frac{1}{BN} \sum_{i=1}^B \sum_{j=1}^N (R(\pi_i^j | w^j)) \nabla_\theta \log_{p_\theta}(\pi_i^j | s_i^j)$ ;
7    $\theta \leftarrow \text{ADAM}(\theta, \nabla L(\theta))$ ;
7 end
```

to the edges with the greatest gap of cost between their upper and lower bounds ensures that the overall scenario is as unfavorable as possible.

Lemma: For the solution x to the min-max problem, the worst-case scenario $\bar{c}_j$, defined as

$$\sum_{j \in N} \bar{c}_j x_j = \min_{c \in U} \sum_{j \in N} c_j x_j,$$

satisfies

$$\bar{c}_j = c_j^* + \delta_j \gamma_j,$$

where $\bar{\gamma}_j$ is given by

$$\bar{\gamma}_j = \begin{cases} 1, & j = j_1, j_2, \dots, j_\Gamma, \\ 0, & j = j_{\Gamma+1}, \dots, j_n, \end{cases}$$

and the sequence $(j_1, j_2, \dots, j_n)$ represents the non-increasing order of the values $\{\delta_j x_j\}$.

D. Solution Indication

If we change the objective f_2 from the relative robustness form to the absolute RTSP distance, i.e. $f_1^{wc}(x)$, for the aggregation function $F(x)$ of a MOP, the uncertain and deterministic parts of the RTSP can be separated as:

$$\begin{aligned} F(x) &= w_1 f_1(x) + w_2 (f_1(x) + f_1^{wc}(x) - f_1(x)) \\ &= f_1(x) + w_2 h(x) \end{aligned} \quad (9)$$

Algorithm 3: Worst-case of Tour Length f^{wc}

Input: Original edge distance $\bar{d}$, worst-case edge distance $\underline{d}$, and the degree of uncertainty (budget Γ).

Output: f^{wc} .

```
1 for  $m \leftarrow 1$  to  $M$  do
2    $\xi_i^m \leftarrow \text{Top}(\Gamma^m, \bar{d}_{ij}^m - \underline{d}_{ij}^m)$ ,  $i = 1, \dots, n$ ;
3    $f_m^{wc} \leftarrow f_m^o + \sum_{i=1}^n \xi_i^m$ ;
4 end
5 return  $f^{wc}$ ;
```

where $h(x, \Gamma, d) = f_1^{wc}(x) - f_1(x)$ refers to the extra uncertain part. In the new environment, the degree of uncertainty disturbance has changed, which is reflected in variations in the uncertainty budget Γ or the cost interval $[\underline{d}, \bar{d}]$. The objective function of the new problem can be expressed as:

$$f'(x) = f_1(x) + h(x, \Gamma', d') \quad (10)$$

We believe that there exists a similarity between the new problem and the subproblems already trained in our multiobjective model. Since the effect of Γ on the function $h(x, \Gamma, d)$ is monotonic and approximately linear, and the difference between d and d' is easy to estimate, we can adjust w^j such that the j 'th subproblem is close to $f'(x)$. Thus, we can indicate the optimal solution of $f'(x)$ by means of the solution set of the j 'th subproblem.

This adjustment process can be achieved by training a fine-tuned neural network. For a given training instance, we use the multiobjective model to obtain the optimal solutions of the subproblems and compute their fitness in different environments. The optimal solution represented by w is then output as the label, enabling the inference of the relationship between Γ and Γ' .

IV. COMPUTATIONAL EXPERIMENTS

In this section, the experimental settings are first provided. Then, the experimental results on the comparison of MO-RCO with other existing baselines are reported and analyzed.

A. Experimental Settings

1) *Compared Algorithms:* Benders Decomposition (BD) is an exact approach introduced by Montemanni et al. [15]. Simulated Annealing (SA) is a local search based heuristic along with a simulated annealing implementation introduced by Lu et al. [17]. We would like to thank the authors of [19] for providing the source codes they reproduced. We have slightly modified the above approaches to make them applicable to the problem under the min-max criterion. Genetic Algorithms (GA) and Ant Colony Optimization (ACO) are two heuristics implemented in PlatEMO [24]. For the MOP, we use MOEA/D [22] and DRL-MOA [6] as baselines. We have incorporated a worst-case total length calculation method similar to **Algorithm 3** to facilitate the computation of the RTSP.

2) *Test Instances:* The test instances in this paper include two types of data. In type I, the coordinates of the cities are randomly generated in $[0, 1] \times [0, 1]$. The lower bound $\underline{d}_{ij}$ of cost depends on the Euclidean distance between cities i and j , while the upper bound $\bar{d}_{ij}$ is randomly chosen from the interval $[1.5\underline{d}_{ij}, 2.5\underline{d}_{ij}]$. This setting can be considered a simplified version of the second type of instance setting in [15]. In type II, the lower bound $\underline{d}_{ij}$ and the upper bound $\bar{d}_{ij}$ are randomly generated within the interval $[0, 100]$ and $[100, 200]$ as in [18]. Note that uncorrelated uncertain instances are usually harder to solve. For different n , we generate 10 instances for each type.

For the RTSP with 2 objectives, we independently generate another set of random coordinates and repeat the cost generation procedure as in Type 1. The number of subproblems is set to 101, and the reference point is set as $(n + \Gamma_1, n + \Gamma_2)$.

3) *Parameter Settings:* We follow most of the settings in DRL-MOA [6]. The model is trained on a single RTX 3070 GPU with 120,000 different instances. We use the Adam optimizer with learning rate η of 10^{-4} and batch size of 200. The only distinction is that, for efficiency, we limited the training to 3 epochs instead of 5. For GA and ACO, the maximum iteration limit is set to 100,000, and for SA, the computation time is set to $15n$.

B. Experimental Results

Table I summarizes the average optimality gap of 10 generated instances obtained by BD, ACO, GA, SA, and our method. The following observations can be obtained from this table:

- Whether in general scenarios or complex situations, our method is able to obtain high-quality solutions in few seconds.
- In terms of solution quality, compared to the best method BD for the optimization problem, our method maintains a gap of less than 2% for smaller-scale problems, and for larger-scale problems, the gap remains smaller than some heuristic methods.
- In terms of runtime, our method clearly consumes less time than the comparing algorithms, significantly reducing the computational time cost.

Figure 2 shows the PFs obtained by DRL-MOA, MOEA/D and our method. Table II summarizes the results of instances with different nodes. In this experiment, in DRL-MOA we just use the original model and ignore possible uncertainty, and in MOEA/D we consider the worst-case scenario of each objective. The following observations can be obtained from them:

- MO-RCO outperforms the original DRL-MOA, which suggests the importance of considering the robustness when dealing with MOPs with uncertainty.
- The solutions of MO-RCO shows better diversity in all instances. This echoes the reasoning in III-A objective, demonstrating the importance of trade-off consideration in uncertainty scenarios and the effect of setting internal weights $\{\lambda_k\}$, which differ from the weight vector w .

TABLE I: The average optimality gap of 10 generated instances obtained by five algorithms.

Problem	BD		ACO		GA		SA		MO-RCO	
	Gap	Time/s	Gap	Time/s	Gap	Time/s	Gap	Time/s	Gap	Time/s
20-I	0.00%	1800	4.06%	34.27	0.86%	4.32	1.60%	150.3	0.77%	0.41
30-I	0.00%	1800	8.03%	53.14	3.14%	4.93	0.98%	450.3	0.69%	0.68
40-I	0.00%	1800	11.08%	73.46	5.17%	5.79	2.35%	600.3	1.84%	0.85
60-I	0.00%	1800	16.92%	113.93	15.51%	7.63	12.98%	900.4	5.21%	1.17
20-II	0.00%	1800	3.47%	43.07	1.54%	4.03	2.15%	150.3	1.82%	0.43
30-II	0.00%	1800	6.71%	65.1	3.79%	5.20	2.78%	450.3	1.64%	0.77
40-II	0.00%	1800	11.85%	96.83	6.65%	6.52	4.76%	600.3	3.08%	0.92
60-II	0.00%	1800	17.68%	249.27	16.13%	8.80	12.43%	900.4	8.74%	1.26

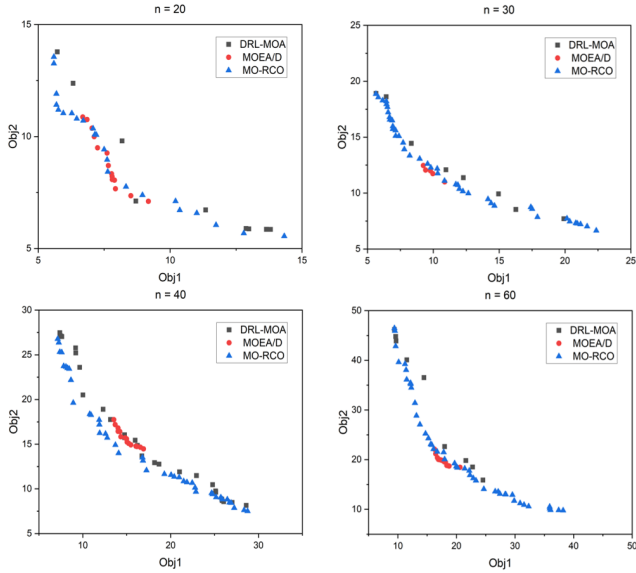


Fig. 2: Plot of the PFs obtained by three algorithms in solving random multiobjective instances with different nodes.

TABLE II: The hypervolume (HV) values of 10 generated multiobjective instances obtained by three algorithms.

Problem	DRL-MOA		MOEA/D		MO-RCO	
	HV	Time	HV	Time	HV	Time
20	317	1.77	304	11.34	323	2.83
30	823	2.24	718	12.09	848	3.80
40	1506	2.93	1186	12.81	1521	5.24
60	3621	4.15	3064	14.67	3729	9.81

Table III and Figure 3 present the average optimality gap relative to BD for models trained with different nodes. The following observations can be obtained from them:

- Models trained on a specific number of nodes demonstrate a distinct advantage in solving problems of the corresponding scale. When addressing problems with varying node counts, those trained with the same number of nodes consistently deliver superior solutions.
- Models trained with a higher number of nodes show small gap for problems with fewer nodes, which highlights the strong generalization ability of our models. However, when models trained with fewer nodes are applied to problems with a higher number of nodes, the quality of the solutions progressively degrades as the problem scale increases.

TABLE III: The average optimality gap against BD of 10 instances obtained by models trained with different nodes. The values 20, 30, 40, 60 in parentheses indicate that the model was trained with the settings of $n = 20, 30, 40, 60$, respectively.

Problem	MO-RCO(20)	MO-RCO(30)	MO-RCO(40)	MO-RCO(60)
20	0.77%	1.07%	0.86%	1.31%
30	1.62%	0.69%	1.46%	0.82%
40	7.62%	4.61%	1.95%	3.86%
60	13.60%	9.03%	7.36%	5.21%

Table IV summarizes the results of instances obtained by BD and MO-RCO with two budget settings in training. The following observations can be obtained from this table:

- Compared to BD, models trained under a specific budget exhibit similar gaps when solving problems with different budgets. This further highlights the model's robust generalization capability across varying budgets, suggesting that training separate models for different budget levels is unnecessary.

TABLE IV: The results of instances obtained by BD and MO-RCO with two budget settings in training. The values 0.25 and 0.5 in parentheses indicate that the model was trained with the settings of $\Gamma = 0.25n$ and $\Gamma = 0.5n$, respectively.

Problem	BD			MO-RCO (0.25)			MO-RCO (0.5)		
	Obj	Gap	Time/s	Obj	Gap	Time/s	Obj	Gap	Time/s
$\Gamma = 0$	5.11	0.00%	1800	5.27	3.29%	0.76	5.26	3.01%	0.75
$\Gamma = 0.25n$	7.35	0.00%	1800	7.53	2.39%	0.78	7.58	3.13%	0.78
$\Gamma = 0.5n$	8.78	0.00%	1800	9.02	2.72%	0.79	9.04	3.05%	0.82
$\Gamma = 0.75n$	9.74	0.00%	1800	10.01	2.67%	0.82	10.04	3.03%	1.12
$\Gamma = n$	10.23	0.00%	1800	10.52	2.91%	0.84	10.49	2.62%	1.19

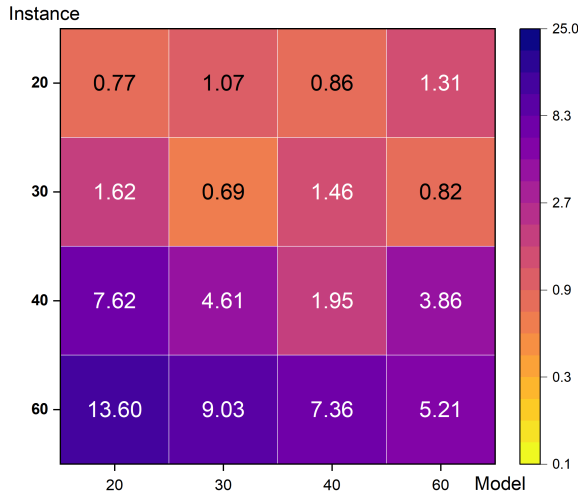


Fig. 3: The optimality gap (%) of instances with different nodes obtained by models trained with different nodes.

- Furthermore, models trained with different budgets show no significant differences in results. Therefore, it is sufficient to select an appropriate budget for training without the need for multiple budget-specific models.

In summary, MO-RCO demonstrates superior performance in solving the RTSP, whether in the context of single-objective or multi-objective optimization. The model is generalizable to both the number of nodes and the degree of uncertainty, while also enabling decision-makers to balance optimality and robustness effectively.

V. CONCLUSION

To solve TSP involving cost uncertainty, we proposed a DRL framework in this paper, denoted by MO-RCO. It transforms a RCO problem into a MOCO problem and solve the problem in an end-to-end way. MO-RCO provides a set of solutions that balance optimality and robustness, and also predicts solutions for problems with new uncertainty parameters. We also discussed the worst-case of RTSP with budget uncertainty. Additionally, we introduce the prototype of

a multiobjective RCO problem and demonstrate the superiority of MO-RCO in solving such problems. Our experimental results show that MO-RCO outperforms other methods on instances with varying budgets at different nodes. In the future, we will explore more types of RCO problems and conduct a more detailed analysis of multiobjective RCO problems.

REFERENCES

- [1] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.
- [2] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–15.
- [3] E. B. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
- [4] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–25.
- [5] X. Lin, Z. Yang, and Q. Zhang, "Pareto set learning for neural multi-objective combinatorial optimization," in *Int. Conf. Learn. Represent.*, 2022.
- [6] K. Li, T. Zhang, and R. Wang, "Deep reinforcement learning for multiobjective optimization," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3103–3114, Jun. 2021.
- [7] H. Li, Y. Tang, Y. Shui, and J. Sun, "A Q-learning evolutionary multiobjective framework for multiobjective optimization with separable and interacting variables," in *2024 IEEE Congress on Evolutionary Computation (CEC)*, Yokohama, Japan, 2024, pp. 1–8.
- [8] Z. Wang, S. Yao, G. Li, and Q. Zhang, "Multiobjective combinatorial optimization using a single deep reinforcement learning model," *IEEE Trans. Cybern.*, vol. 54, no. 3, pp. 1984–1996.
- [9] J. M. Mulvey, R. J. Vanderbei, and S. A. Zenios, "Robust optimization of large-scale systems," *Oper. Res.*, vol. 43, no. 2, pp. 264–281, 1995.
- [10] A. Ben-Tal and A. Nemirovski, "Robust convex optimization," *Math. Oper. Res.*, vol. 23, no. 4, pp. 769–805, 1998.
- [11] D. Bertsimas and M. Sim, "The price of robustness," *Oper. Res.*, vol. 52, no. 1, pp. 35–53, 2004.
- [12] P. Kouvelis and G. Yu, *Robust Discrete Optimization and Its Applications*, vol. 14. Springer Science & Business Media, 2013.
- [13] H. Aissi, C. Bazgan, and D. Vanderpooten, "Min-max and min-max regret versions of combinatorial optimization problems: A survey," *Eur. J. Oper. Res.*, vol. 197, no. 2, pp. 427–438, 2009.
- [14] A. L. Soyster, "Technical note - Convex programming with set-inclusive constraints and applications to inexact linear programming," *Oper. Res.*, vol. 21, no. 5, pp. 1154–1157, 1973.
- [15] R. Montemanni, J. Barta, M. Mastrolilli, and L. M. Gambardella, "The robust traveling salesman problem with interval data," *Transp. Sci.*, vol. 41, no. 3, pp. 366–381, 2007.
- [16] A. Chassein and M. Goerigk, "On the recoverable robust traveling salesman problem," *Optim. Lett.*, vol. 10, pp. 1479–1492, 2016.

- [17] C.-C. Lu, S.-W. Lin, and K.-C. Ying, "Minimizing worst-case regret of makespan on a single machine with uncertain processing and setup times," *Appl. Soft Comput.*, vol. 23, pp. 144–151, 2014.
- [18] K. Hasegawa and W. Wu, "A heuristic approach for the robust traveling salesman problem," in *2022 IEEE Int. Conf. Ind. Eng. Eng. Manag. (IEEM)*, pp. 561–565, 2022.
- [19] P. Xiao, Z. Zhang, J. Chen, J. Wang, Z. Zhang, "Neural combinatorial optimization for robust routing problem with uncertain travel times," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024.
- [20] A. Ben-Tal, A. Goryashko, E. Guslitzer, and A. Nemirovski, "Adjustable robust solutions of uncertain linear programs," *Math. Program.*, vol. 99, no. 2, pp. 351–376, 2004.
- [21] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [22] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, 2007.
- [23] J. Zhang, W. Wu, and M. Yagiura, "A heuristic approach for the robust traveling salesman problem," in *2017 IEEE Int. Conf. Ind. Eng. Eng. Manag. (IEEM)*, 2017.
- [24] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "Platemo: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Comput. Intell. Mag.*, vol. 12, no. 4, pp. 73–87, 2017.